



CENTRO UNIVERSITÁRIO MÁRIO PALMÉRIO
CURSO CIÊNCIA DA COMPUTAÇÃO

Paulo Cesar Soares

Pesquisa e Relatório Teórico

Monte Carmelo

2026

1 - O que é uma Thread e qual a diferença entre processo e thread.

Uma **thread** (ou linha de execução) é a menor unidade de processamento que um sistema operacional pode gerenciar e executar dentro de um processo.

Enquanto um processo é um programa em execução completo com seus próprios recursos isolados, uma thread existe *dentro* de um processo e compartilha o mesmo espaço de memória e os arquivos desse processo com outras threads irmãs.

Aspectos	Processo	Thread
Memória	Possui seu próprio espaço de memória.	Compartilha a memória do processo.
Criação	Normalmente exige mais recursos.	Normalmente é mais leve.
Falha	Uma falha costuma ficar isolada no processo.	Uma falha pode afetar outras threads do processo.
Comunicação	Precisa de mecanismos próprios para trocar dados.	Pode acessar os mesmos objetos.

2 - As duas formas de criar uma Thread em Java (estender a classe Thread e implementar a interface Runnable) — vantagens, desvantagens e quando usar cada uma.

Java oferece duas formas tradicionais de criar uma tarefa concorrente. A primeira é estender a classe Thread. A segunda é implementar a interface Runnable.

Forma	Vantagens	Desvantagens	Quando usar
Estender Thread	É direta e fácil de entender em exemplos pequenos.	A tarefa fica ligada à Thread e a classe perde a possibilidade de herdar outra classe.	Em exemplos simples ou quando a própria classe precisa representar uma thread.
Implementar Runnable	Separa a tarefa da execução, permite reutilização e preserva a herança da classe.	É necessário criar um objeto Thread para executar a tarefa.	Na maioria dos projetos, principalmente quando a classe representa uma tarefa.

Exemplo com Thread:

```
class      MinhaThread      extends      Thread      {
public      void      run()      {
System.out.println("Tarefa      em      execução");
}
}
```

```
new MinhaThread().start()
```

Exemplo com Runnable:

```
class MinhaTarefa implements Runnable {  
    public void run() {  
        System.out.println("Tarefa em execução");  
    }  
}
```

```
Thread thread = new Thread(new MinhaTarefa());  
thread.start();
```

Runnable costuma ser a melhor escolha porque deixa claro que uma classe descreve o trabalho e outra classe controla a thread que executa esse trabalho.

3 - O ciclo de vida de uma Thread (estados: New, Runnable, Blocked, Waiting, Timed Waiting, Terminated).

Durante sua execução, uma thread passa por estados definidos pela enumeração `Thread.State`. O estado muda quando a thread é iniciada, espera algum evento, tenta obter um bloqueio ou termina sua tarefa.

Estado	Explicação simples	Exemplo
NEW	A thread foi criada, mas start ainda não foi chamado.	<code>new Thread(tarefa)</code>
RUNNABLE	Está pronta para executar ou está executando.	Código dentro de <code>run</code>
BLOCKED	Espera conseguir entrar em uma área <code>synchronized</code> .	Duas threads disputam o mesmo monitor
WAITING	Espera por outra ação, sem tempo definido.	<code>join</code> sem tempo limite
TIMED WAITING	Espera por um período definido.	<code>Thread.sleep(1000)</code>
TERMINATED	O método <code>run</code> terminou.	Fim da tarefa

Depois de criada, a thread fica em NEW. A chamada a `start` leva a thread para RUNNABLE. Durante a execução, ela pode esperar ou ficar bloqueada. Quando `run` termina, a thread passa para TERMINATED e não pode ser iniciada novamente.

4 - Métodos fundamentais: `start()`, `run()`, `sleep()`, `join()` e a diferença entre chamar `start()` e chamar `run()` diretamente.

Método	Função
start	Inicia uma nova thread e faz a JVM chamar run nessa nova linha de execução.
run	Contém o código da tarefa que será executada pela thread.
sleep	Pausa a thread atual por um tempo definido em milissegundos.
join	Faz uma thread esperar o término de outra thread.

Diferença entre start e run

A chamada a start cria uma nova linha de execução. A JVM agenda essa nova thread e depois chama run. Quando run é chamado diretamente, nenhuma nova thread é criada. O método funciona como uma chamada comum e é executado pela thread atual.

5 - Condições de corrida (race conditions): o que são, um exemplo prático e por que ocorrem.

Uma condição de corrida ocorre quando duas ou mais threads acessam o mesmo dado e o resultado depende da ordem em que elas executam. Como essa ordem pode mudar a cada execução, o erro pode aparecer apenas algumas vezes.

Um exemplo simples é um contador compartilhado. A operação contador++ parece ser uma única ação, mas ela envolve ler o valor, somar um e gravar o resultado.

```
int contador = 0;
contador++; // leitura, soma e escrita
```

Se o contador vale 10, duas threads podem ler 10 ao mesmo tempo. As duas calculam 11 e gravam 11. O resultado correto deveria ser 12, mas uma atualização é perdida. O problema ocorre porque existe um dado compartilhado que pode mudar e não há sincronização para controlar o acesso.

6 - Mecanismos de sincronização em Java: a palavra-chave synchronized (métodos e blocos) e, opcionalmente, classes do pacote java.util.concurrent (como AtomicInteger ou ReentrantLock).

A palavra-chave synchronized permite que somente uma thread por vez execute uma parte protegida do código usando o mesmo objeto como monitor. Essa parte recebe o nome de seção crítica.

Método synchronized

```
public synchronized void incrementar() {
    contador++;
}
```

Nesse caso, todo o método fica protegido. É uma forma simples de sincronizar quando todas as operações do método trabalham com o mesmo dado compartilhado

Bloco synchronized

```
synchronized (objetoCompartilhado) {  
    objetoCompartilhado.atualizar();  
}
```

O bloco protege apenas as instruções necessárias. Isso pode reduzir o tempo de bloqueio e deixar outras partes do método livres para execução concorrente.

Recurso	Uso principal	Observação
synchronized	Proteger métodos ou blocos que usam dados compartilhados.	O bloqueio é liberado automaticamente ao sair do bloco.
AtomicInteger	Fazer operações atômicas com números inteiros.	É útil para contadores simples.
ReentrantLock	Controlar o bloqueio de forma mais explícita.	unlock deve ser chamado em um bloco finally.

A sincronização deve proteger somente o estado realmente compartilhado. Usar bloqueios em excesso pode reduzir o desempenho e também criar deadlocks, situação em que duas threads ficam esperando uma pela outra.

7 - Conclusão pessoal: principais dificuldades e aprendizados durante a pesquisa.

Aprendi a diferença entre start() e run() também me confundiu. Depois da pesquisa, entendi que start() inicia uma nova thread, enquanto chamar run() diretamente apenas executa o método na thread atual.

Também aprendi para que servem sleep() e join(). O sleep() pausa a thread por um tempo, enquanto o join() faz uma thread esperar que outra termine.

E também quando várias threads alteram a mesma informação ao mesmo tempo, o programa pode apresentar resultados errados. O synchronized ajuda a evitar esse problema, permitindo que apenas uma thread por vez execute a parte protegida do código